

Population Census & Demographic Insights Analytics System

An End-to-End Data Analytics Application using Python, Databricks, Apache Airflow, and Power BI.

ABSTRACT

Government agencies and urban planners face significant challenges in managing and interpreting massive, fragmented census datasets to drive regional development. Although vast amounts of demographic data are collected, the lack of automated, high-performance processing systems often results in data silos and delayed insights into critical social indicators.

This project presents an end-to-end Census Data Engineering & Demographic Analytics System that processes large-scale population data to provide real-time insights into literacy, gender ratios, and employment trends. The system is implemented using a scalable Medallion Architecture (Bronze, Silver, Gold) on Azure Databricks, with robust workflow orchestration managed through Apache Airflow in a dockerized environment.

Strategic demographic indicators and regional benchmarks are visualized through Power BI dashboards using the Databricks Connector, enabling policymakers to monitor urbanization trends, detect literacy gaps at the district level, and plan evidence-based infrastructure interventions. The solution demonstrates professional data engineering, transformation, and visualization practices applied to the public policy and governance domain.

Table of Contents

S. No	Contents	Page No
1.	Business Context & Problem Statement	3
2.	Project Objectives	4
3.	Technology Stack	4
4.	Dataset Description	5
5.	Dataflow	7
6.	System Architecture & Workflow	8
7.	Execution for Workflow Automation: From Databricks Setup to Airflow Trigger	12
8.	Visualization & Reporting	20
9.	Challenges Faced & Solutions	25
10.	Strategic Use Cases	26
11.	Limitations & Future Enhancements	27
12.	Conclusion	28

1. Business Context & Problem Statement

National census bureaus and government planning departments collect vast amounts of demographic data from multiple sources, including regional surveys, administrative registries, and civil databases. However, these massive datasets are often fragmented across disparate formats and processed through legacy systems, resulting in delayed insights into the socio-economic health of the population.

The key challenges faced by these organizations include:

- **Data Fragmentation:** Demographic and geographic information is often distributed across multiple regional and national systems without a unified structure.
- **Slow Processing at Scale:** Traditional manual analysis and legacy tools fail to process millions of records efficiently, leading to outdated insights.
- **Inconsistent Data Quality:** Lack of standardized cleaning processes for geographic names (States/Districts) and handling of null values leads to inaccurate reporting.
- **Delayed Strategic Insights:** Absence of real-time, actionable dashboards prevents policymakers from identifying critical gaps in literacy, employment, and gender parity as they emerge.

Due to these limitations, government interventions and budget allocations are often reactive rather than proactive. This project aims to address these challenges by building a unified Population Census and Demographic Insights Analytics & Engineering Platform that leverages modern cloud architecture to support evidence-based governance and strategic national planning.

2. Project Objectives

The primary objectives of this project are:

- To design and implement an end-to-end population census analytics pipeline.
- To ingest, clean, and transform raw national demographic datasets using Python and PySpark.
- To develop a multi-layered Medallion Architecture (Bronze, Silver, Gold) for structured data refinement and auditability.
- To calculate critical socio-economic indicators such as literacy rates, gender ratios, and employment trends across diverse geographic scales.
- To automate complex data workflows and dependency management using Apache Airflow and Databricks Workflows.
- To visualize demographic KPIs and regional disparities through interactive Power BI dashboards.
- To demonstrate real-world data engineering practices including cloud-based big data processing, containerization, and scalable ETL orchestration.

3. Technology Stack

Python, Pandas, NumPy, PySpark, Databricks, Apache Airflow, Power BI, GitHub

4. Dataset Description

The system utilizes four primary datasets representing diverse dimensions of national demographic information. The project processes a combined volume of 620,000+ records to simulate a large-scale census operation. Intentional data quality issues, such as inconsistent geographic naming conventions, null values, and duplicate entries, were introduced to demonstrate the robust cleaning and transformation capabilities of the Silver Layer.

Dataset	Approximate Record Count	Description
Region	180+ Records	Maps State and District codes to geographic names and administrative levels.
Population	120,000+ Records	Contains core demographic counts, age-group distributions, and gender ratios.
Literacy	200,000+ Records	Tracks educational attainment, literacy levels, and schooling status by region.
Employment	300,000+ Records	Records workforce participation, occupational sectors, and employment status.

4.1.1 Region Dataset

Description:

It contains official state and district identifiers along with their names and administrative classification (state or union territory). This dataset acts as a master reference and is used to validate regional identifiers in other datasets.

Key Use Cases in Project:

- Regional validation and consistency checks
- Joining population, literacy, and employment data
- Enabling region-wise analytics and comparisons

4.1.2 Population Dataset

Description:

This dataset records population counts by state and district, segmented by year, age group, gender, ethnic group, and area type (total, rural, urban). It provides the foundation for demographic and growth analysis.

Key Use Cases in project:

- Population growth analysis
- Age and gender distribution trends
- Rural vs urban population comparison

4.1.3 Literacy Dataset

Description:

The literacy dataset contains population counts classified by literacy status (Literate or Illiterate), along with age group, gender, ethnic group, year, and area type. It is used to compute literacy rates and identify regional disparities.

Key Use Cases in Project:

- Literacy rate calculation
- State and district-level literacy comparison
- Correlation analysis with employment and population

4.1.4 Employment Dataset

Description:

This dataset records employment counts by State and District, categorized by age group, gender, ethnic group, employment status, and economic sector (Primary, Secondary, Tertiary, and Not in Labour Force). Special handling is applied for age groups not eligible for employment.

Key Use Cases in Project:

- Employment rate analysis
- Sector-wise employment distribution
- Workforce planning and policy insights

5. Dataflow



6. System Architecture & Workflow

This section outlines the end-to-end journey of the census data, from initial ingestion to final visualization.

6.1 Architectural Framework

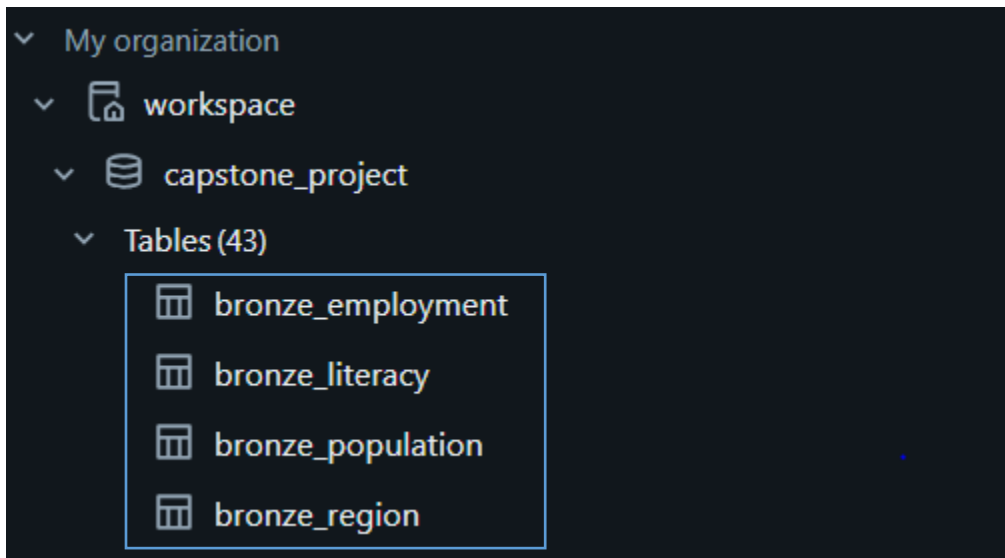
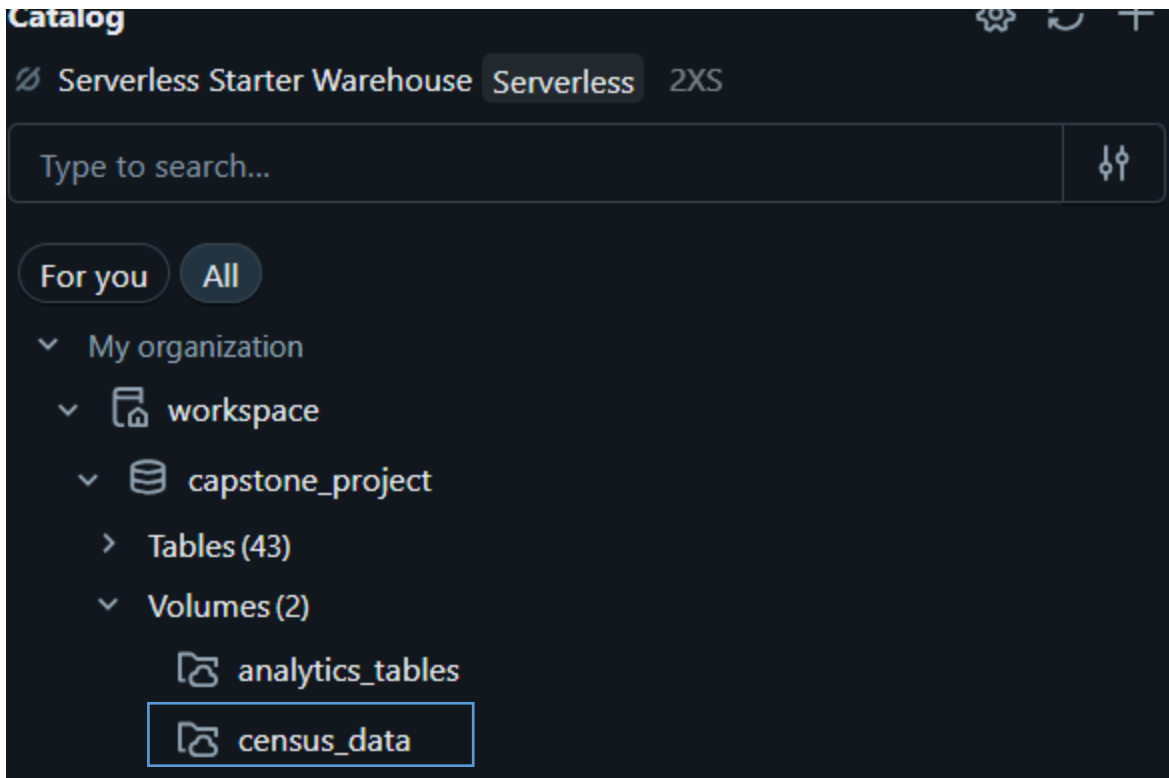
The platform utilizes a modern, decoupled architecture designed for high-performance data processing and scalable reporting:

- **Azure Databricks:** Serving as the central compute engine for large-scale data engineering (PySpark) and analytical modeling.
- **Apache Airflow:** Acting as the external orchestrator to manage task dependencies, scheduling, and error-handling via Directed Acyclic Graphs (DAGs).
- **Power BI:** Functioning as the presentation layer for executive-level dashboards and regional drill-down analytics.

6.2 The Step-by-Step Workflow

Step 1: Raw Data Ingestion (Bronze Layer)

- **Source Connectivity:** Source census data in CSV format is loaded into Databricks-managed cloud storage.
- **High-Fidelity Loading:** Data is ingested into Delta tables using Spark, preserving the 1:1 original schema of the source files.
- **Auditability:** At this stage, no transformations or business logic are applied. This ensures a historical "snapshot" is maintained for troubleshooting, data lineage, and the ability to re-process raw data if logic changes downstream.



Step 2: Cleaning and Conforming (Silver Layer)

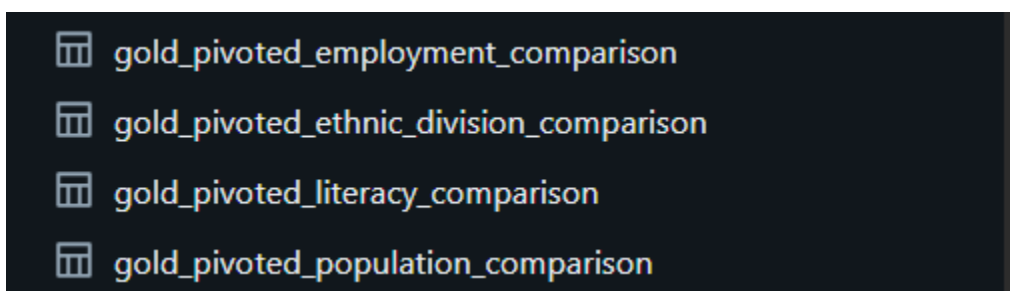
- **Quality Enforcement:** Spark jobs remove duplicates, handle missing values, and standardize geographic naming conventions (e.g., State/District aliases).
- **Year-Based Aggregation:** Data is grouped by Census Year/Year-Wise to enable time-series analysis.

- **Filing for Analytics:** The result is a set of conformed "Silver" tables that serve as the single source of truth for the entire enterprise.



Step 3: Business Analytics (Gold Layer)

- **Metric Calculation:** Final aggregations are performed to create specific KPIs (e.g., Literacy Rates, Urbanization Ratios, and Employment Trends), both State and District wise.
- **Optimization:** Tables are optimized for DirectQuery performance in Power BI, ensuring low-latency interactive reporting.





```
gold_district_pivoted_employment_comparison
gold_district_pivoted_ethnic_division_comparison
gold_district_pivoted_literacy_comparison
gold_district_pivoted_population_comparison
```

Step 4: Workflow Orchestration

- Unified Pipeline Execution: Databricks Workflows manages the sequential execution of the data lifecycle. This ensures that the Bronze → Silver → Gold transition is strictly governed, with built-in validation checks at each stage to prevent downstream data contamination.
- External Orchestration & Monitoring: Apache Airflow serves as the master controller, triggering the Databricks jobs and providing a centralized "single pane of glass" for monitoring performance, managing retries, and logging execution metadata.
- Two DAG files were created and triggered. One comprises of a single job pipeline where all the above 3 tasks are under a single job and the file name is single.py with dag id: population_census_single_job.
- The another DAG file is to trigger three different job, one for each layer. The name of the file is multi.py and dag id: populationCensus_sequentialJob.

Step 5: Insights & Visualization

- **Semantic Layer Integration: Power BI** connects directly to the curated **Gold tables** in Databricks, ensuring a consistent business logic layer for all reporting.
- **Actionable Intelligence:** Dashboards are designed to translate complex census metrics into visual action points, allowing stakeholders to identify demographic trends and regional disparities through high-interactivity features like slicers and drill-throughs.

7. Execution for Workflow Automation: From Databricks Setup to Airflow Trigger

The following steps outline the end-to-end process of configuring the jobs and orchestrating them via Airflow.

Step 1: Databricks Job Configuration

Before triggering from Airflow, the jobs must be defined within the Databricks Workspace:

- Create Notebooks: Develop three distinct notebooks for the Bronze, Silver, and Gold layers using PySpark logic.
- Configure Workflows:
 - Single Job: Create one Databricks Job containing three tasks: Bronze->Silver->Gold linked by dependencies. Note the Job ID. (single.py)
 - Sequential Jobs: Create three separate Databricks Jobs (one for each layer). Note each individual Job ID to be used in the multi.py DAG.
- Generate Access Token: Go to User Settings ->Developer ->Access Tokens and generate a PAT (Personal Access Token) for Airflow authentication.

Step 2: Airflow Environment Setup (Docker)

Airflow is managed within a containerized environment to ensure consistency.

1. Place DAG Files: Move single.py and multi.py into the ./dags folder of the Airflow project directory.
2. Initialize Containers: Open your terminal in the project folder and run:
docker-compose up -d
3. Verify Services: Ensure all containers (webserver, scheduler, worker) are healthy:
docker ps

Step 3: Configuring the Databricks Connection

1. Access the Airflow UI at <http://localhost:8080>.
2. Navigate to Admin -> Connections.
3. Add a new connection with ID databricks_default,
 - Connection Type: Databricks
 - Host: The workspace URL, it can be found in SQL Warehouse -> Connection Details -> Hostname
 - Password: Paste the generated Personal Access Token.

Step 4: Triggering and Monitoring the DAGs

Once the connection is active, you can manage the two orchestration strategies:

- Strategy A: Single Job Pipeline (single.py)
 - Locate the DAG population_census_single_job.
 - Unpause the DAG and click the Trigger button.
 - This sends a single API call to Databricks to run the entire multi-task workflow.

Code for single.py:

```

from airflow import DAG

from airflow.providers.databricks.operators.databricks import DatabricksRunNowOperator

from airflow.operators.python import PythonOperator

from datetime import datetime, timedelta

# Default arguments for the dag

default_args = {

    "owner": "krishnapriya_capstone",

    "depends_on_past": False,

    "retries": 1,

    "retry_delay": timedelta(minutes=5),

}

# Helper function to log the databricks run_id via xcom

def log_run_id(ti):

    run_id = ti.xcom_pull(

        task_ids="trigger_databricks_census_job",

        key="run_id"

    )

    print(f"Databricks job triggered successfully. run_id = {run_id}")

```

```

with DAG(
    dag_id="population_census_single_job",
    default_args=default_args,
    start_date=datetime(2026, 1, 1),
    schedule="@daily",
    catchup=False,
    tags=["census", "databricks", "capstone"],
) as dag:

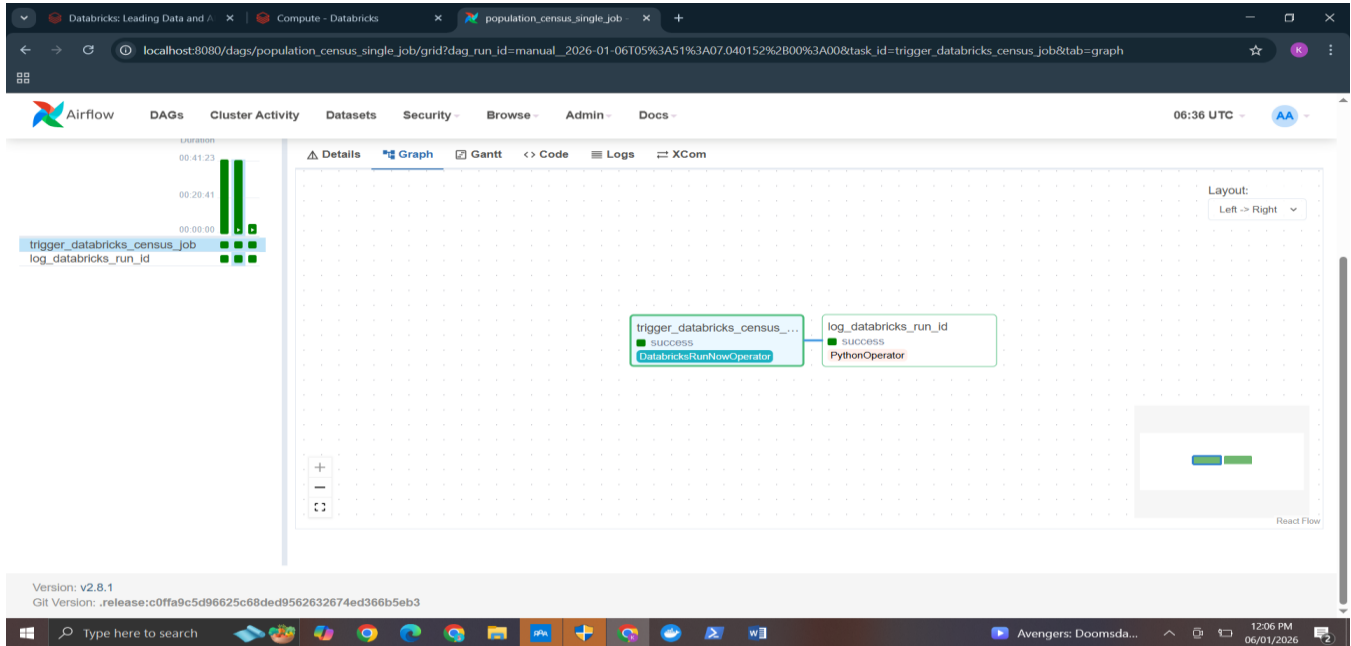
    trigger_census_pipeline = DatabricksRunNowOperator(
        task_id="trigger_databricks_census_job",
        databricks_conn_id="databricksDefault",
        job_id=969233964957203,
        wait_for_termination=False
    )

    log_runid = PythonOperator(
        task_id="log_databricks_run_id",
        python_callable=log_run_id
    )

    trigger_census_pipeline >> log_runid

```

population_census_single_job output:



- Strategy B: Sequential Job Pipeline (multi.py)
 - Locate the DAG `populationCensus_sequentialJob`.
 - Triggering this DAG will execute three separate tasks in Airflow. Airflow will manage the sequence: it will wait for the Bronze Job to finish before triggering the Silver Job, and so on.

Code for multi.py:

```
from airflow import DAG
```

```
from airflow.providers.databricks.operators.databricks import DatabricksRunNowOperator
```

```
from airflow.operators.empty import EmptyOperator
```

```
from datetime import datetime, timedelta
```

1. Default arguments

```
default_args = {  
    "owner": "krishnapriya_capstone",
```

```

    "depends_on_past": False,

    "retries": 1,

    "retry_delay": timedelta(minutes=5),
}

```

2. DAG Definition

```

with DAG(

    dag_id="populationCensus_sequentialJobs",

    default_args=default_args,

    start_date=datetime(2026, 1, 1),

    schedule=None, # Manual trigger (best for Databricks pipelines)

    catchup=False,

    tags=["census", "bronze", "silver", "gold", "databricks"],

)

as dag:

    # Start marker

    start_pipeline = EmptyOperator(

        task_id="start_pipeline"

    )

    # Bronze Layer

    bronze_layer = DatabricksRunNowOperator(

        task_id="run_bronze_layer",

        databricks_conn_id="databricksDefault",

        job_id=672530452226296,

```

```

    wait_for_termination=True
)

# Silver Layer

silver_layer = DatabricksRunNowOperator(
    task_id="run_silver_layer",
    databricks_conn_id="databricksDefault",
    job_id=1042163527863242,
    wait_for_termination=True
)

# Gold Layer

gold_layer = DatabricksRunNowOperator(
    task_id="run_gold_layer",
    databricks_conn_id="databricksDefault",
    job_id=498960847987735,
    wait_for_termination=True
)

# End marker

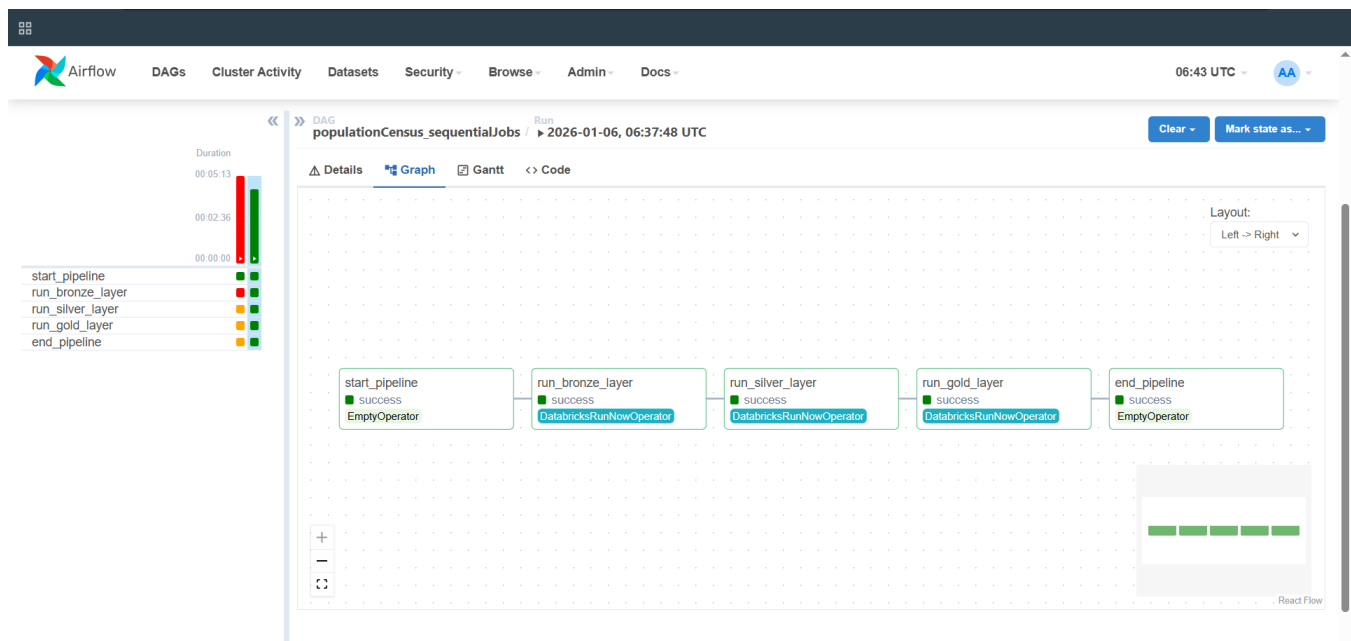
end_pipeline = EmptyOperator(
    task_id="end_pipeline"
)

# 3. Sequential dependency

start_pipeline >> bronze_layer >> silver_layer >> gold_layer >> end_pipeline

```

populationCensus_sequentialJob output:



Job Runs Triggered by Airflow:

Jobs & Pipelines

▼ Create new

 **Ingestion pipeline**
Ingest data from apps, databases and files

 **ETL pipeline**
Build ETL pipelines using SQL and Python

 **Job**
Orchestrate notebooks, pipelines, queries and more

Jobs & pipelines

Job runs

Send feedback

Filter by name or ID sub...

AllJobsPipelinesOwned by meAccessible by meFavoritesTagsRun asCreate

Name	Type	Tags	Run as	Trigger	Recent runs	
Bronze_Layer	Job		krishnapriyakakuma...		— — — ✓ ✓	▶ ✎ ⋮
Gold_Layer	Job		krishnapriyakakuma...		— — — ✓ ✓	▶ ✎ ⋮
Silver_layer	Job		krishnapriyakakuma...		— — — ✓ ✓	▶ ✎ ⋮
SingleJob_MultiTask	Job		krishnapriyakakuma...		✓ ✓ ✓ ✓ ✓	▶ ✎ ⋮
Workflow_automation	Pipeline		krishnapriyakakuma...		— — — — ✓	▶ ✎ ⋮

< Previous

Next >

8. Visualization & Reporting

8.1 State-Wise Demographic Insights

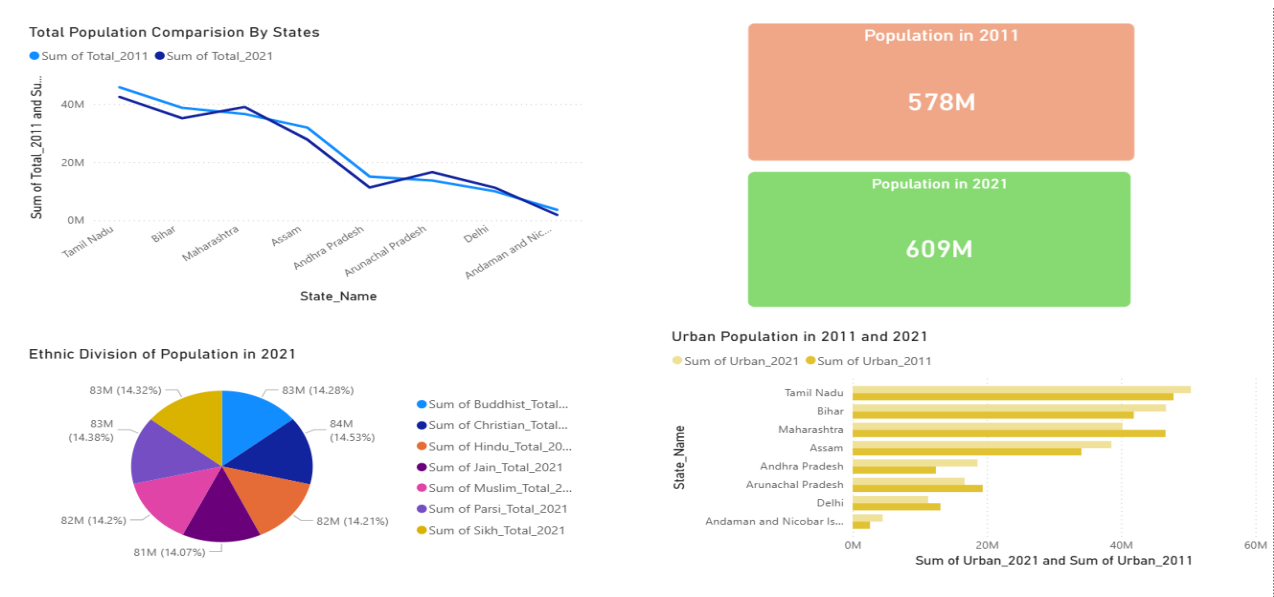
Focus: Comparative analysis of population growth, ethnic distribution, and urbanization across major states between 2011 and 2021.

Key Questions Answered:

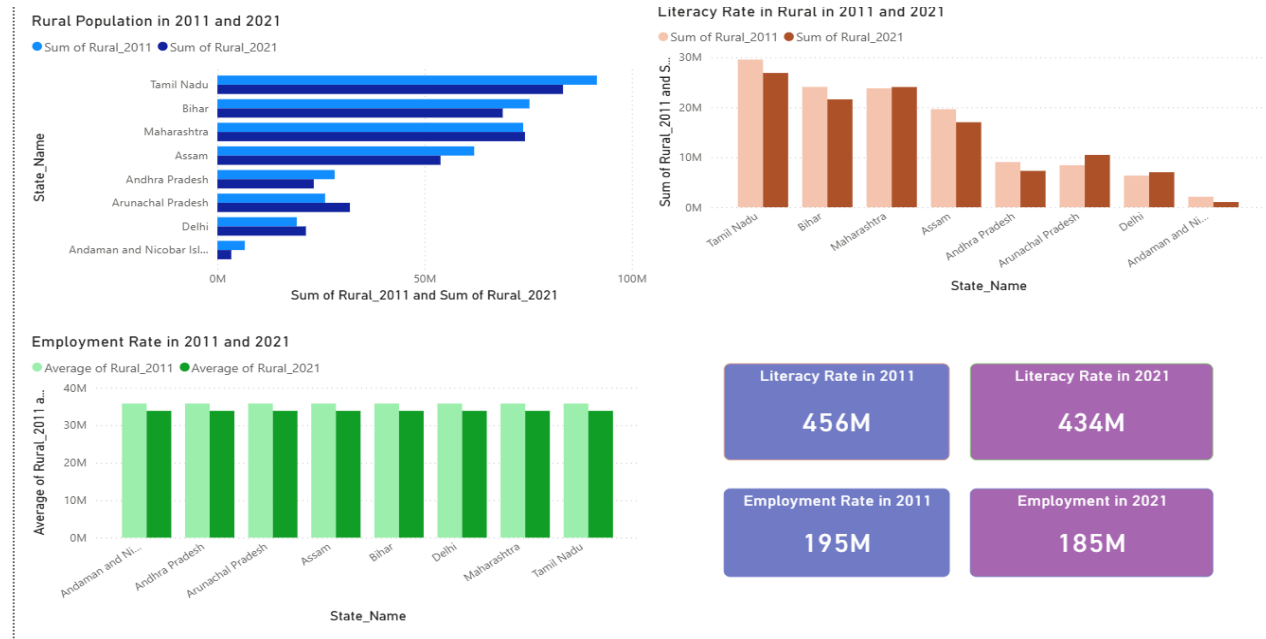
- What is the total population trajectory? Card visuals show a growth from 578M in 2011 to 609M in 2021.
- Which states are leading in urbanization? Bar charts compare rural vs. urban splits, identifying states like Tamil Nadu and Maharashtra as high-density hubs.
- What is the religious/ethnic makeup? A pie chart breaks down the population into Buddhist, Christian, Hindu, Jain, Muslim, Parsi, and Sikh segments for 2021.
- How did literacy and employment rates vary between two year and in rural and urban regions?

Why this page matters: It provides the big picture for national policymakers to allocate federal resources and understand high-level migration toward urban centers.

State-Wise Dashboard-1



State-Wise Dashboard-2



8.2 District-Wise Analytics

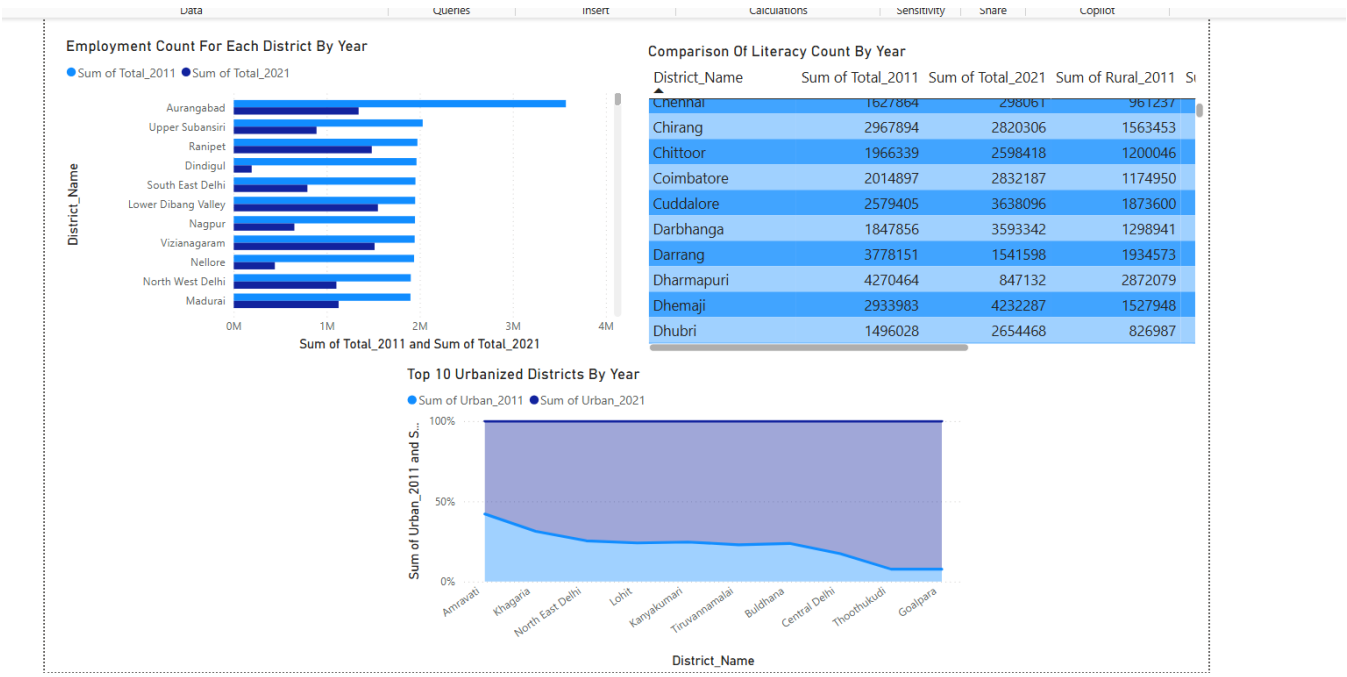
Focus: Localized reporting that allows for district-level "drill-downs" into employment and urbanization.

Key Questions Answered:

- Which districts are the most urbanized? An area chart highlights the "Top 10 Urbanized Districts," showing the percentage shift in urban living.
- What are the specific literacy numbers for a local area? A detailed matrix table provides exact counts for districts like Chennai, Chittoor, and Coimbatore, comparing 2011 vs. 2021 data.
- Where is the highest employment concentration? Bar charts track employment counts for specific districts such as Aurangabad and Nagpur.

Why this page matters: This is the most "actionable" layer for local governance, enabling district collectors to plan for specific utilities, schools, and healthcare facilities based on precise local counts.

District-Wise Dashboard



8.3 Silver Aggregates: Literacy & Employment Trends

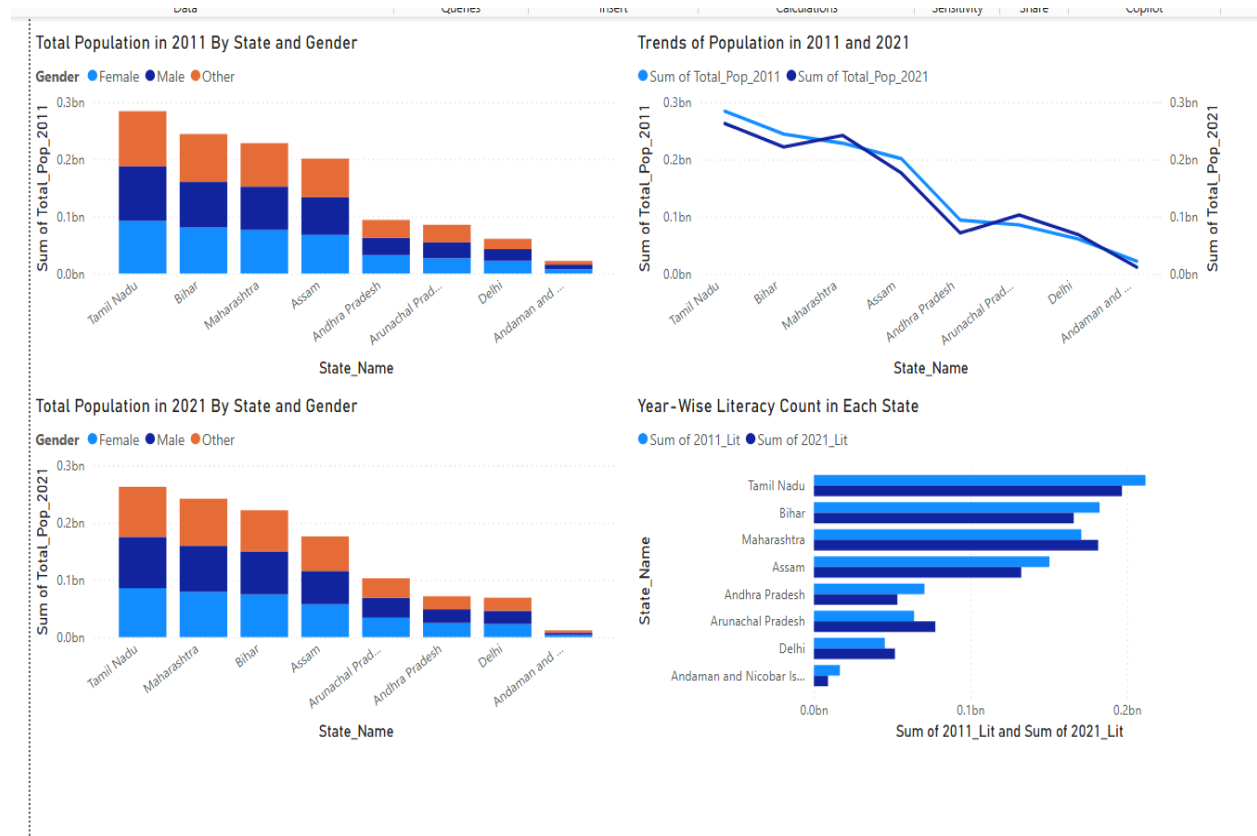
Focus: Deep-dive into socio-economic indicators, specifically focusing on the correlation between gender, education, and work.

Key Questions Answered:

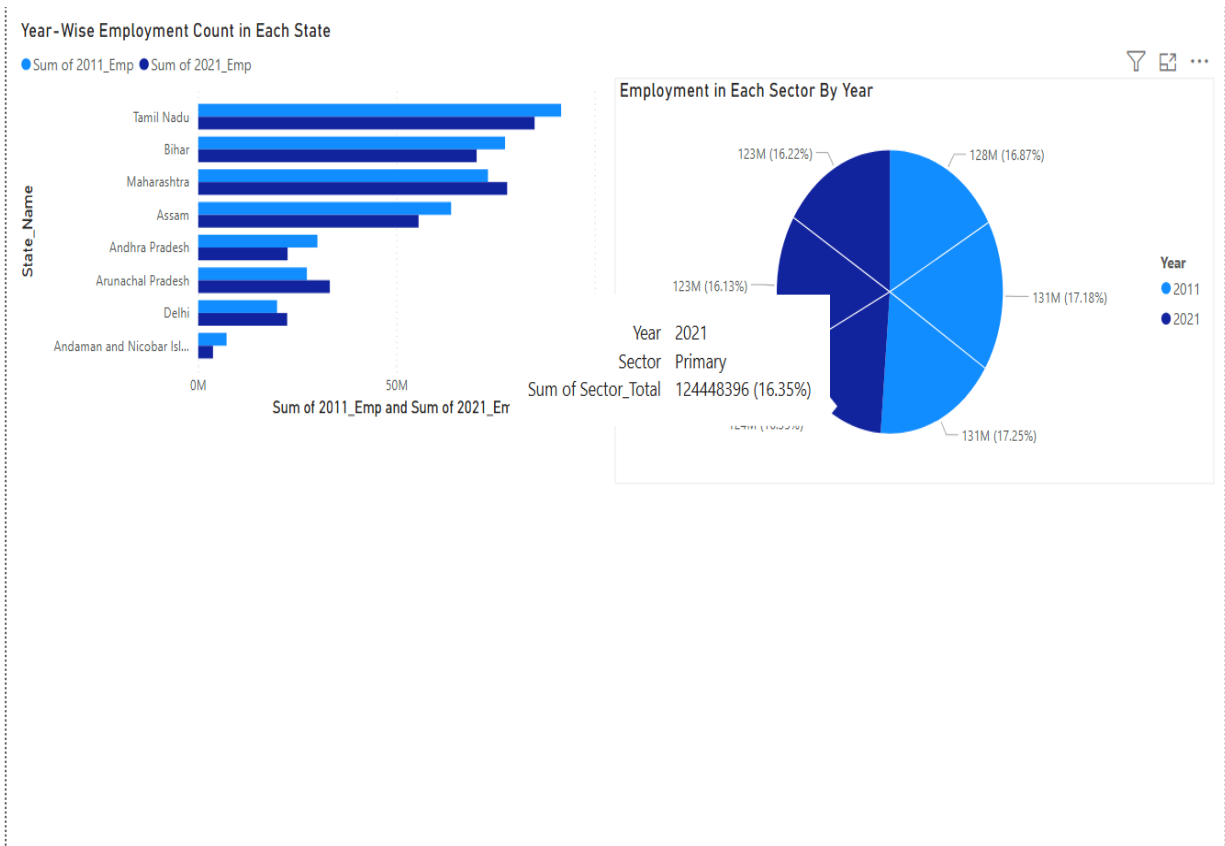
- How has literacy improved over the decade? Horizontal bar charts track the "Year-Wise Literacy Count," showing significant gains in states like Bihar and Assam.
- What is the workforce participation by sector? Pie and bar charts visualize employment counts, allowing users to filter by Primary, Secondary, or Tertiary sectors across different years.
- Is there a gender gap in population growth? Stacked column charts analyze population distribution by Male, Female, and Other across states for both census cycles.

Why this page matters: This dashboard validates the "Silver Layer" transformations, proving that cleaned data can reveal critical gaps in workforce readiness and educational access.

Silver Aggregates Dashboard-1



Silver Aggregates Dashboard-2



9. Challenges Faced & Solutions

Some of the challenges faced are:

- Scalability of Data Processing: Leveraged PySpark on Azure Databricks to process 620,000+ records, overcoming memory overhead issues found in traditional environments.
- Workflow Coordination: Used Apache Airflow and Databricks Workflows to automate the Bronze-Silver-Gold sequence, preventing manual errors and data corruption.
- Geographic Inconsistency: Resolved mismatched state and district names in the Silver Layer using a standardized region dataset.
- Airflow-Docker Connectivity: Fixed authentication and CSRF errors between Airflow containers and the Databricks API by configuring Personal Access Tokens (PAT).
- Schema Evolution: Managed structural differences between 2011 and 2021 datasets using Delta Lake's schema enforcement to maintain data integrity.

10. Strategic Use Cases

10.1 Evidence-Based Governance

- Enables national and regional policymakers to identify critical literacy gaps and gender imbalances across diverse districts.
- By utilizing Power BI Treemaps and comparative metrics, agencies can move away from "one-size-fits-all" policies toward targeted funding and social welfare programs for underprivileged demographics.

10.2 Infrastructure & Resource Optimization

- Analyzes granular population density, urbanization trends, and age-group distributions to inform urban planning.
- Supports the strategic placement of essential public services—such as hospitals, schools, and public transport routes—ensuring infrastructure development aligns with actual population needs rather than estimates.

10.3 Data Integrity & Scalability for Large-Scale Operations

- Provides a standardized, automated, and audit-ready pipeline designed to ingest and refine millions of records with high fidelity.
- This framework can be rapidly adapted for other nationwide data needs, such as economic forecasting, tax revenue analysis, or crisis management, ensuring the government maintains a "Single Source of Truth."

10.4 Disaster Response & Public Health Resilience

- Integrates district-level population density with demographic vulnerability factors (e.g., concentrations of elderly populations or children).
- In the event of a pandemic or natural disaster, emergency services can use the Gold Layer metrics to identify "High-Risk Zones," enabling proactive vaccine distribution or targeted evacuation plans based on real-time capacity and demographic risk profiles.

11. Limitations & Future Enhancements

Few limitations of this project include:

- Real-Time Processing: The pipeline is currently batch-oriented; implementing Structured Streaming would allow for real-time administrative data ingestion.
- Automated Alerting: Integrating Slack or Email notifications into the Airflow DAG would proactively alert stakeholders when KPIs cross critical thresholds.
- Data Integration: Expanding the model to include climate and infrastructure spending data would provide a more comprehensive socio-economic view.

12. Conclusion

In conclusion, this project successfully demonstrates the implementation of a scalable, end-to-end data engineering ecosystem that transforms raw, fragmented census data into high-value strategic insights. By integrating a Medallion Architecture on Azure Databricks with Apache Airflow orchestration, the system ensures data integrity through rigorous cleaning in the Silver layer and high-performance analytics in the Gold layer. The resulting Power BI dashboards provide a unified "Single Source of Truth," enabling policymakers to transition from reactive decision-making to proactive, evidence-based governance across literacy, employment, and urbanization sectors. Ultimately, this solution bridges the gap between complex big data processing and actionable public policy, offering a versatile framework that can be adapted for diverse large-scale demographic and socio-economic challenges.